

Lehrveranstaltung Mechatronische Systeme und Produkte - Kompendium -



Ansprechpartner:
M. Sc. Lukas Rausche
lukas.rausche@kit.edu

Stand: 02.11.2021

Inhaltsverzeichnis

1. Wichtige Hinweise	4
2. Matlab mit Code Composer Studio verbinden	5
2.1. Installationsreihenfolge	5
2.2. Simulink konfigurieren	5
2.3. Installation Nexonar Motion Visualizer	6
3. Hardware	7
3.1 Der Mikrocontroller	7
3.2 Netzteil	8
3.3 Digitale Eingänge	8
3.4 Analoge Eingänge	8
3.5 Digitale Ausgänge	8
3.6 Gesamtplatine	9
3.7 FT-Taster	12
3.8 FT-Glühlampe	12
3.9 FT-Motor ohne Encoder	12
3.10 FT-Motor mit Encoder	13
3.11 FT-Farbsensor	13
3.12 FT-Spurensensor	14
3.13 FT-Fototransistor	14
3.14 Sharp Distanzsensoren 21YK und 31SK	15
3.15 FT-Akku	18
3.16 Carson-Akku	18
3.17 Pololu Encoder Motor	18
3.18 Nexonar Tracker	19
4. Programmierung	22
4.1 Simulink Modell	22
4.2 Beispielprogramm I	22
4.3 Programme auf den Mikrocontroller spielen	23
4.4 Stateflow	23
4.5 Elemente eines Zustandsautomaten	24

4.6 Verbindung des Simulink Modells mit dem Automaten	26
5. Software-Module	26
5.1 ADC-Kennlinie	26
5.2 Winkelmessung mit einem Pololu Encoder-Motor	27
5.3 Rate Transition	27
5.4 P-Regelung	28
5.5 Kommunikation mit anderen Systemen	28

1. Wichtige Hinweise

Auf dieser Seite sind einige sehr wichtige Verhaltensregeln zur Verwendung der Hard- und Software zusammengefasst. Lesen Sie deshalb diese Seite gut durch, verstehen sie die angesprochenen Punkte und behalten Sie diese während des Arbeitens im Auge!

- ⚠ Bevor Änderungen an der Platine oder den Verbindungen zu den Sensoren und Aktoren vorgenommen werden, muss die Spannungsversorgung über das Netzteil, den Akku oder die USB-Verbindung unterbrochen werden!
- ⚠ Verbinden Sie niemals den Akku, das Netzteil, den 3V, den 5V oder den 9V Ausgang der Platine direkt mit dem GND-Anschluss (*Ground*, Masse) der Platine oder eines Sensors oder Aktors.
- ⚠ Sind Motoren an die Platine angeschlossen, sollte die 9V Motorspannungsversorgung solange ausgeschaltet bleiben bis erfolgreich ein Programm mit Drehzahlbegrenzung (PWM) auf den Mikrocontroller geladen wurde. Ansonsten liegt an jedem Motortreiber der HIGH-Pegel an und die Motoren drehen mit maximaler Drehzahl.
- ⚠ Die Spannungsversorgung wird erst hergestellt, wenn der fehlerfreie Anschluss aller Verbindungen sowie die vollständige Isolierung der Kabel überprüft wurden.
- ⚠ Beide Distanzsensoren von Sharp dürfen an VCC nur mit 5V versorgt werden!
- ⚠ Trennen Sie Sensoren und Aktoren Vorsichtig von den Verbindungskabeln! Ziehen Sie dabei nicht am Kabel, sondern an der Hülse und am Fischertechnikstecker.
- ⚠ Öffnen Sie nicht die Platine!
- ⚠ Ein häufiger Fehler ist:

*The call to idelink_ert_make_rtw_hook, during the exit hook generated the following error:
Error while building the project.*

Erstellen, speichern und führen Sie deshalb ihre Simulink-Programme aus dem Standard MATLAB-Ordner oder einem Ordner mit sehr kurzer Pfadlänge aus (z.B. vom Desktop).

- ⚠ Löschen Sie vor einer Fehlersuche die Ordner *s/prj* sowie **_rtw* und kompilieren Sie das Programm neu.
- ⚠ Verändern Sie keine Einstellungen der Hardware oder der Software des Kamerasystem-PCs, der Infrarot-Kameras, der Tracker oder des WLAN-Routers für das Kamerasystem!
- ⚠ Lassen sie im Entwurfsprozess die verwendeten Kabel ausreichend lang. Eine schöne Verbindung der Kabel kann am Ende des Workshops erfolgen.
- ⚠ Sie können Markierungen, kleine Schilder mit Bezeichnungen, an den Kabeln anbringen um den Anfang und das Ende zu kennzeichnen, dadurch sparen Sie sich Arbeit und Zeit bei der Verkabelung und können die Bezeichnung sowohl in SysML als auch in ihrem Matlab-Programm wiederverwerten.
- ⚠ Benutzen Sie das Isolierband um Verbindungen zwischen den Bundhülsen und Fischertechnik-Steckern oder zwischen zwei Fischertechnik-Steckern vor Kurzschlüssen zu schützen.

2. Matlab mit Code Composer Studio verbinden

Damit die in **Matlab&Simulink** erstellten Programme mit den Sensoren und Aktoren kommunizieren können, muss das Programm in eine Sprache übersetzt werden, welche der Mikrocontroller versteht. Hierfür wird das **Code Composer Studio v10 (CCS10)** verwendet. Der in diesem Kapitel vorgestellte Arbeitsschritt sollte bereits auf den PCs in den Laboren durchgeführt worden sein. Für die Einrichtung ihres privaten PCs oder Laptops gehen Sie das folgende Kapitel Schritt für Schritt durch. Verwenden Sie ausschließlich die nachfolgenden Programme.

Software	Version
Matlab&Simulink	2021b 64-bit
Code Composer Studio v10	10.2.0.00009

2.1. Installationsreihenfolge

Alle für die Installation notwendigen Programme, außer Matlab (siehe [Softwareshop des KIT](#)), stehen in ILIAS zum Download zur Verfügung. Speichern Sie diese lokal ab. Sie benötigen für alle notwendigen Installationen bis zu **60 GB** Festplattenspeicher. Gehen Sie die folgenden Schritte durch:

1. Installieren Sie Matlab 2021b 64bit
 - Hierfür benötigen Sie einen Matlab User-Account
 - Installieren sie **alle** Toolboxes
2. Installieren Sie das Code Composer Studio, wählen Sie „Custom Installation“ und wählen Sie dort „C2000 real time hardware“ aus.
3. Installieren Sie „controlsuiteinstaller“ mit „Full Installation“
4. Installieren Sie „setup_TI_Header“
5. Installieren Sie „C2000Ware_3_03_00_00_setup“
6. Starten Sie Matlab **als Administrator**
 - Installieren Sie das Addon „MATLAB Support for MinGW-w64 C/C++ Compiler“
 - Installieren Sie das Addon „Embedded Coder Support Package for Texas Instruments C2000 Processors“
 - Während der Installation **alle Prozessoren** auswählen, alle Programme welche automatisch installiert werden können automatisch installieren und anschließend die Installationspfade der vorher installierten Programme angeben

Starten Sie zum Testen der Installation das Programm *Blinklicht* in Simulink (siehe Kapitel 4). Bei erfolgreicher Installation blinkt die LED an **D_OUT 1**

2.2. Simulink konfigurieren

Um eine korrekte Konfiguration aller Hardware-Parameter sicherzustellen, verwenden Sie immer die bereitgestellte Datei „VorlageMatlab2021b.slx“ (oder eine Kopie davon) als Vorlage und arbeiten

innerhalb dieses Modells. Nur so kann sichergestellt werden, dass alle Hardwareeinstellungen korrekt sind.

2.3. Installation Nexonar Motion Visualizer

Der *Nexonar Motion Visualizer* ist diejenige Software, welche die Rohdaten der Infrarot-Kameras ausliest, vorverarbeitet und mit weiterer Software und Hardware die Positionsdaten auf dem Mikrocontroller bereitstellt (siehe Kapitel 5.5). Die Installation dieser Software auf weiteren PC's wird nur dafür verwendet, um die Helligkeitsstufen der Nexonar Tracker anzupassen (siehe Kapitel 3.18). Es genügt also, wenn der Gruppen-PC in den Laboren diese Software installiert hat.

Die Nexonar *SDK* sowie die Software *Motion Visualizer* stehen in ILIAS zum Download bereit.

Zur Installation der Software wird das NET Framework 4.7.1 sowie ein C++ Compiler benötigt. Bei einer bereits vorhandenen Installation einer oder mehrere dieser Komponenten kann es zu denselben Problemen kommen wie [hier](#) beschrieben. Wurde die Installation gemäß Kapitel 2.1 für Matlab 2021b 64bit bereits durchgeführt, kann die Software in der folgenden Reihenfolge installiert werden:

1. SDK → setup.exe
2. Motion Visualizer → setup.exe

Lässt sich das Programm anschließend nicht starten, sind die Schritte in [link](#) beschrieben durchzuführen, damit das Net Framework 4.7.1 sowie ein C++ Compiler fehlerfrei installiert werden.

3. Hardware

Im folgenden Teil dieses Kompendiums wird ein Überblick über die im Workshop verwendete Hardware gegeben. Dazu zählen die Fischertechnik (FT), Sharp, Nexonar, Polulu und Texas Instruments (TI) Sensoren und Aktoren.

3.1 Der Mikrocontroller

Der Mikrocontroller ist zusammen mit dem Experimenter-Board (TI F28379D LaunchPad) von Texas Instruments in Abbildung 1 dargestellt.

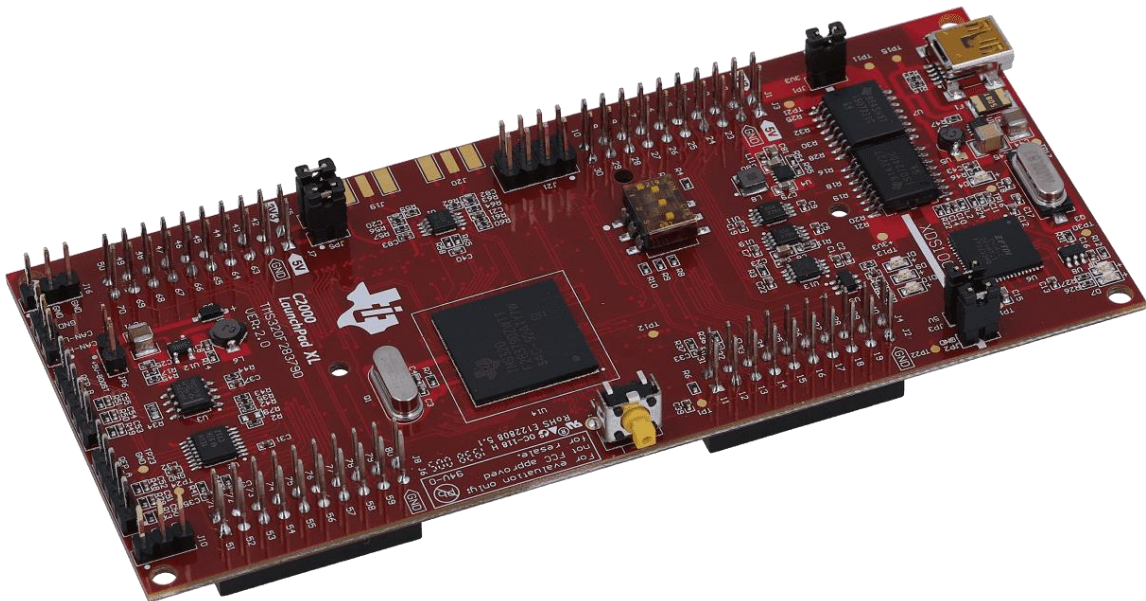


Abbildung 1: Experimenter-Board mit dem Mikrocontroller C2000 von Texas Instruments

- ⚠ Der Mikrocontroller ist kein echtzeitfähiges System! Es wird keine Warnung ausgegeben, wenn die Rechenressourcen überschritten wurden. D.h. im Falle eines zu aufwendigen Programmes entsteht eine Latenz und anschließende Programmteile werden verzögert ausgeführt.

3.2 Netzteil

Das für die Spannungsversorgung verwendete Netzteil ist in Abbildung 2 dargestellt. Es handelt sich um ein MeanWell GST60A09-P1J.



Abbildung 2: MeanWell Netzteil

3.3 Digitale Eingänge

Die digitalen Eingänge der Gesamtplatine werden als Schnittstelle zur Zwischenschaltung zum Controller verwendet, um die Fischertechnikspannung von 9V auf die vom Mikrocontroller verwendete Spannung von 3.3V herabzusetzen. Dazu wird auf den Komparator an den positiven Eingang das Signal der Fischertechnik-Hardware gegeben. Dieses Signal wird mit dem Referenzpegel am negativen Eingang verglichen. Ist das Eingangssignal größer als der Referenzpegel, wird der Ausgang der Schaltung auf 3.3V geschaltet, ansonsten auf 0V. An diese Eingänge werden alle Sensoren und Taster angeschlossen, welche ein digitales Signal ausgeben: **FT-Taster**, **FT-Fototransistor**, **FT-Spursensor**, und **FT-Encoder**.

3.4 Analoge Eingänge

Die analogen Eingänge werden verwendet um analoge Signale in den Controller einzulesen. Die maximale Eingangsspannung die auf die Eingänge gegeben werden sollte liegt bei 5V. Dieses Spannungsniveau wird auf 3.3V abgebildet. Das Übersetzungsverhältnis des linearen Spannungsteilers der Zwischenschaltung zwischen den Eingängen und dem Controller beträgt damit 0,66. Sensoren die ein analoges Signal ausgeben sind: **FT-Farbsensor** und **Sharp Distanzsensor**.

3.5 Digitale Ausgänge

Zur Ansteuerung der Fischertechnik LEDs wird eine Spannung von 9V benötigt. Die dazu vorgesehenen digitalen Ausgänge sind mit **D_OUT 1, 2, 3** und **4** gekennzeichnet. Zwischen diesen Ausgängen und

GND wird die LED angeschlossen. Wird einer dieser Ausgänge über die Software (Simulink) auf logisch 1 geschaltet, leuchtet eine angeschlossene Fischertechnik LED auf. Außerdem zeigen kleine LEDs an jedem der digitalen Ausgänge an, ob der entsprechende Ausgang gerade aktiv ist.

- ⚠ Die LEDs von Fischertechnik brauchen gegenüber den LEDs auf dem Mikrocontroller viel Strom. Der digitale Ausgang hat dementsprechend einen kleinen Innenwiderstand, wodurch Kurzschlüsse mit diesem unbedingt zu vermeiden sind!

3.6 Gesamtplatine

Die Anschlüsse und Schalter der Gesamtplatine sind auf zwei Gehäuseseiten aufgeteilt. Auf der vorderen langen Seite sind alle Anschlüsse zur Verwendung der Aktoren und Sensoren angebracht. Diese sind in Abbildung 3 dargestellt. Die kurze Seite mit den Spannungsanschlüssen und dem Wählschalter ist in Abbildung 4 dargestellt.

Darunter fallen acht frei verfügbare digitale Eingänge **D_IN 1, 2, ... 8**, vier frei verfügbare digitale Ausgänge **D_OUT 1, 2, 3 und 4**, acht analoge Eingänge **A_IN 1, 2, ... 8**, acht Motorausgänge **Motor 1 (1_A & 1_B), 2 (2_A & 2_B), ... 8 (8_A & 8_B)**, zwei Quad-Encoder-Eingänge **QEP A (A1 & A2), B (B1 & B2)**, sowie die benötigten Spannungspegel **3V, 5V und 9V** und Masseanschlüsse **GND**. Wenn die Spannungspegel an den jeweiligen Pins anliegen, leuchtet der Neopixel grün. Andernfalls ist die Spannung getrennt. Die Gesamtplatine ist in einem robusten Kasten aus Acrylglas mit den Maßen 19,4cm x 14cm x 6,9cm untergebracht. Betrachtet man die Länge der Wählschalter und der Ports ergibt sich eine ungefähre Bemaßung von 20cm x 14,2cm x 6,9cm.

- ⚠ Bitte schließen sie nichts an den Port neben dem Neopixel an.



Abbildung 3: Anschlussplatine vorne

Verwendung der Ein- und Ausgänge

Um die Ein- und Ausgänge der Gesamtplatine mit der Software zu verbinden, sind nachfolgend einige Tabellen mit der Vergabe der Pins (GPIO-Ports) innerhalb der Simulink-Umgebung aufgeführt.

⚠ Durch die unveränderliche Verdrahtung ist es nicht möglich die Mikrocontrollerpins ihre Aufgabe frei zuzuordnen.

Digitale Eingänge (D_IN)								
Anschluss-Port (Hardware)	1	2	3	4	5	6	7	8
GPIO Port (Simulink)	2	3	4	5	24	16	124	125
Digitale Ausgänge (D_OUT)								
Anschluss-Port (Hardware)	1	2	3	4				
GPIO Port (Simulink)	32	67	111	22				
Analoge Eingänge (A_IN)								
Anschluss-Port (Hardware)	1	2	3	4	5	6	7	8
ADC Modul/Channel (Simulink)	A/14	C/3	B/3	A/3	C/2	B/2	A/2	B/5
PWM Ausgänge (Motor)								
L/R GPIO Port	131	27	130	26	94	97	123	122
ePWM	4A	4B	5A	5B	6A	6B	1A	1B
Quad-Encoder (QEP)								
eQEP Modul	1	2						
Motoren aktivieren GPIO								
	52							

Bedienung der Schalter

Auf einer der kurzen Seiten des Gehäuses sind zwei Schalter, Anschlüsse für den Akku bzw. das Netzteil und das USB-Kabel angebracht. Diese Seite ist in Abbildung 4 zu sehen.

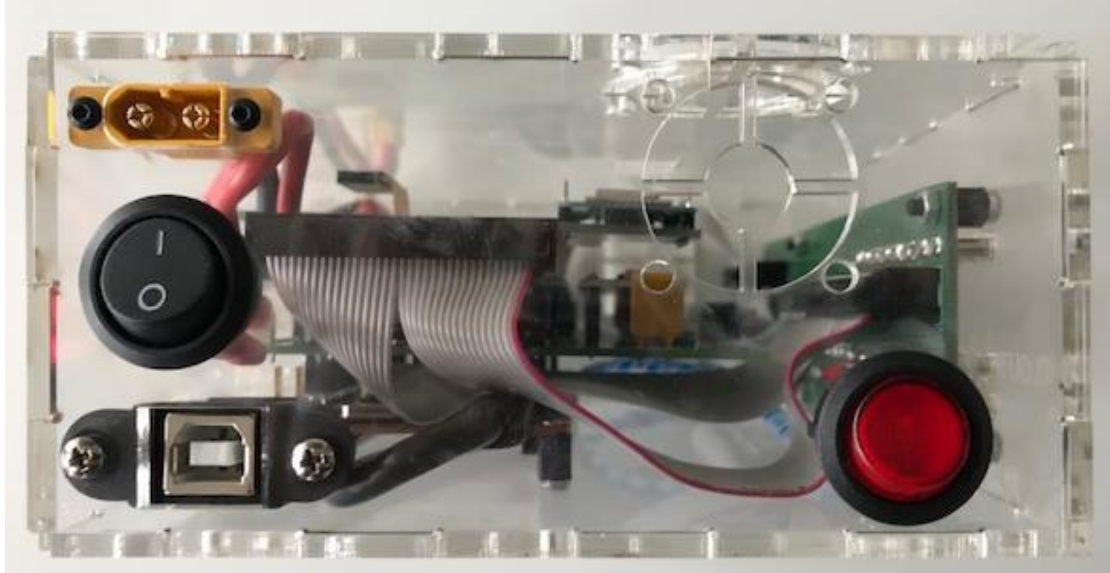


Abbildung 4: Schalter und Anschlüsse der Platine

Der linke Schalter besitzt zwei Stellungen: Oben (I) und unten (O). Ist der Schalter in der oberen Position, wird die Platine mit Strom versorgt. In der unteren Position ist die Platine vom Strom getrennt. Der rechte, rote Schalter hat ebenfalls die Stellungen oben und unten. Mit diesem Schalter können die Motoren (**Motor 1, ..., 8**) mit Strom versorgt werden. In der oberen Stellung ist die Stromversorgung hergestellt, in der unteren getrennt. Des Weiteren leuchtet der rote Schalter, wenn die Motoren über die Software aktiviert wurden (**GPIO Pin 52**). Wenn der Schalter nicht leuchtet, werden die Motoren weder in der oberen noch in der unteren Stellung mit Strom versorgt. Das bedeutet, um die Motoren mit Strom zu versorgen muss sowohl der rote Schalter in der oberen Position sein, als auch über die Software der **GPIO Pin 52** auf 1 gesetzt werden.



Abbildung 5: Gesamtplatine

3.7 FT-Taster

Der FT-Taster kann als Schließer (NO, *normally open*) oder Öffner (NC, *normally closed*) betrieben werden. Dabei ist zu beachten, dass immer der mittlere Anschluss (1) mit 9V und der zweite Anschluss (2 als NC oder 3 als NO) mit einem digitalen Eingang verbunden sind. Der Taster schaltet bei Betätigung zwischen HIGH- und LOW Pegel um. Der Taster kann beispielsweise als Endschalter genutzt werden.

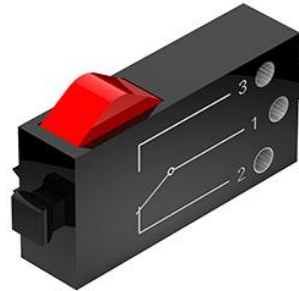


Abbildung 6: Fischer Technik Taster

3.8 FT-Glühlampe

Die Glühlampen von Fischertechnik müssen über einen Digitalen Ausgang und GND verbunden werden. Der benötigte Strom kann nicht durch einen digitalen Ausgang des Mikrocontrollers bereitgestellt werden.

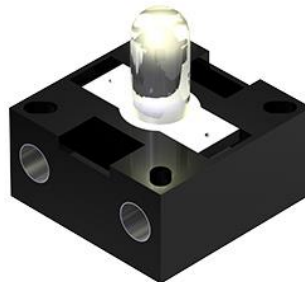


Abbildung 7: Fischer Technik Glühlampe

3.9 FT-Motor ohne Encoder

Dieser Motor besitzt zwei Anschlussklemmen, welche mit einem PWM Ausgang **Motor 1, 2, ... 8** verbunden werden. Die Polung dieser bestimmt die Drehrichtung. Die Drehrichtung kann ebenfalls durch die Software und einen digitalen L/R-Ausgang gewechselt werden. Angesteuert wird der Motor über die PWM Ausgänge des Mikrocontrollers.



Abbildung 8: Fischer Technik Motor ohne Encoder

3.10 FT-Motor mit Encoder

Zusätzlich zum Anschluss für die Stromversorgung, über die Ausgänge **Motor 1, 2, ... 8**, hat dieser Motor noch eine Buchse für ein 3-poliges Anschlusskabel, über das man die Drehbewegung des Motors auswerten kann. Dieses Anschlusskabel muss über das rote (9V) und grüne (GND) Kabel ebenfalls mit Spannung versorgt werden. Das Signalkabel hat die Farbe schwarz und wird mit einem digitalen Eingang verbunden. Die Drehrichtung des Motors kann ebenfalls über die Software und den digitalen L/R-Ausgang verändert werden. Der Encoder dieses Motors erzeugt pro Umdrehung der Motorwelle 75 Impulse.

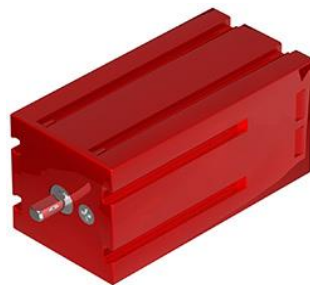


Abbildung 9: Fischertechnik Motor mit Encoder

3.11 FT-Farbsensor

Der Farbsensor hat neben der Spannungsversorgung über 9V und GND noch einen weiteren Anschluss (schwarz) der mit einem analogen Eingang verbunden wird. Je nachdem welche Farbe erkannt wird, liegt eine Spannungsdifferenz zwischen diesem Eingang und GND von 0-9V an. Die konkreten Spannungswerte für Untergründe mit verschiedenen Farben sollten je nach Anforderungsgebiet mit einem Multimeter neu bestimmt werden. Zusätzlich schwanken diese Werte stark mit den Lichtverhältnissen der Umgebung, weshalb durch konstruktive Maßnahmen für annähernd konstante Lichtverhältnisse im Sichtbereich des Sensors gesorgt werden sollte. Beachtet werden sollte das Übersetzungsverhältnis der analogen Eingänge.

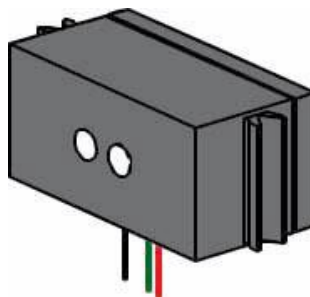


Abbildung 10: Fischertechnik Farbsensor

3.12 FT-Spurensensor

Der Spurensensor (Abbildung 11) hat neben der 9V Spannungsversorgung zwei weitere Signalkabel (gelb und blau). Jedes Signalkabel gibt eine Spannung von 9V aus, wenn im Abstand von 5-30mm von den Sensoren schwarzer Untergrund erkannt wird. Der Liniensensor detektiert also nicht direkt eine Spur, sondern muss durch eine logische Verknüpfung (Software) beider Sensoren ermittelt werden. Beide Anschlüsse werden jeweils an einen digitalen Eingang angeschlossen. In Abbildung 12 ist beispielhaft gezeigt wie einer Spur gefolgt werden kann.

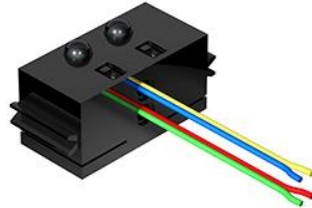


Abbildung 11: Fischertechnik Spurensensor

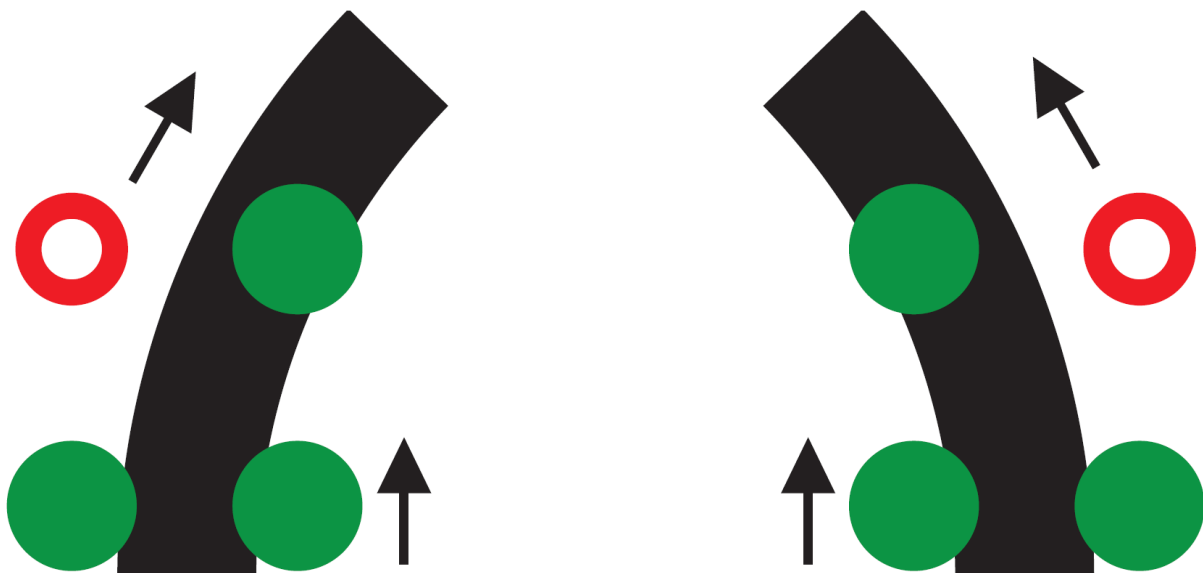


Abbildung 12: Rote Kreise stellen 0V an einem der Signalkabel (blau oder gelb) des Liniensensors dar, keine Linie gemessen. Grüne Flächen entsprechen demnach einer Ausgangsspannung von 9V.

3.13 FT-Fototransistor

Der Fototransistor ist ein Schalter (NO) der auf einfallendes Licht reagiert. Das Halbleiterbauelement wird leitend bei einfallendem Licht auf die Fotodiode. Je höher die Intensität des einfallenden Lichts, also je heller die Umgebung des Bauteils, desto geringer sind der resultierende elektrische Widerstand und die über dem Bauteil anliegende Spannung (Schalter geschlossen, *Eins* am digitalen Eingang). Eine dunkle Umgebung erhöht den elektrischen Widerstand und verringert damit die Leitfähigkeit des Fototransistors (Schalter offen, *Null* am digitalen Eingang). Das in Abbildung 13 dargestellte Bauteil muss an der rot markierten Seite mit 9V und am benachbarten weißen Pin mit einem digitalen Eingang der Gesamtplatine verbunden werden.

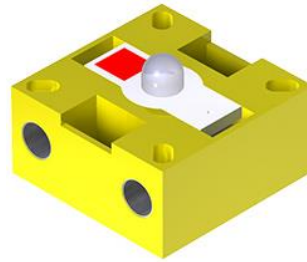


Abbildung 13: Fischertechnik Fototransistor

3.14 Sharp Distanzsensoren 21YK und 31SK

Zur Messung von Abständen im Bereich **4-30cm** sollte der Distanzsensor **41S** und im Bereich **10-80cm** der Distanzsensor **21Y** von Sharp benutzt werden. Beide erkennen Hindernisse und erzeugen aus dem Abstand zu diesen eine analoge Ausgangsspannung zwischen 0-3.2V.

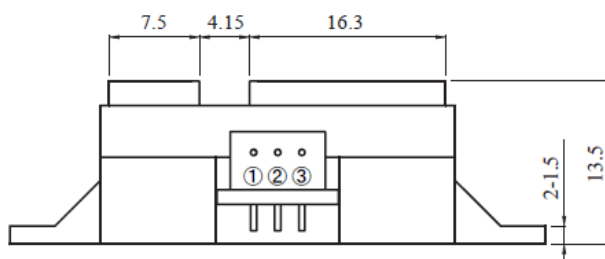
⚠ Beide Sensoren werden mit 5V Spannung versorgt.

Wie in den Abbildungen der nachfolgenden Seiten zu sehen, erzeugen beide Sensoren eine nichtlineare Kennlinie, deshalb muss darauf geachtet werden, in welchem Bereich der Sensor arbeiten soll. Der Sensor wird an einen analogen Eingang angeschlossen. Da die maximale Spannung des Sensors mehr als 3.0V beträgt, wird sie mit dem linearen Spannungsteiler auf den 3.0V Pegel reduziert. Die Funktion der drei Anschlüsse des Sensors sind dem Datenblatt zu entnehmen und in Abbildung 15 zusammengefasst ($V_{cc} = 5V$).

⚠ Trennen Sie ein einmal angeschlossenes Kabel von diesem Sensor nicht wieder ab.



Abbildung 14: Sharp Distanzsensor



Connector signal

	signal name
①	V _o
②	GND
③	V _{cc}

Connector :
J.S.T. TRADING COMPANY, LTD,
S3B-PH

Abbildung 15: Anschlussbelegung Sharp Distanzsensoren

Example of distance measuring characteristics(output)

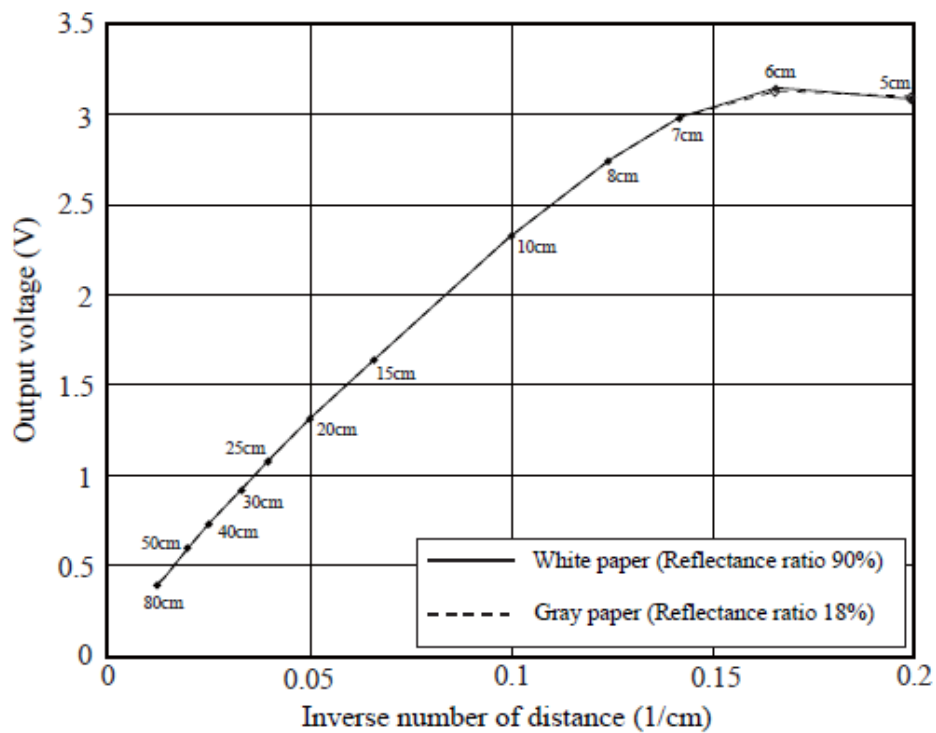
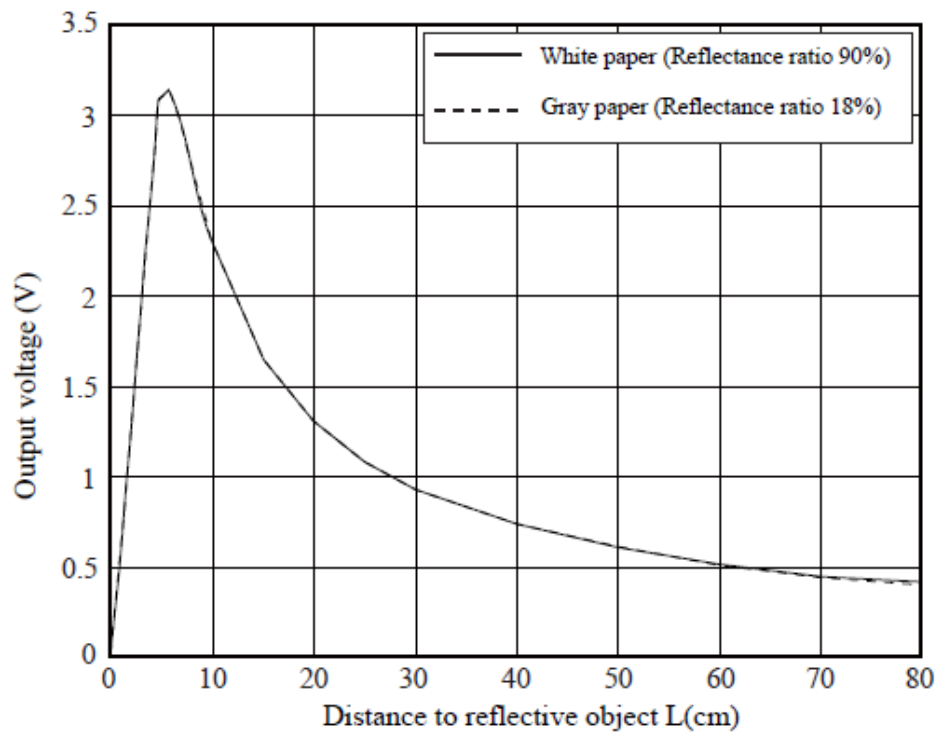


Abbildung 16a: Sharp Distanzsensor 21YK

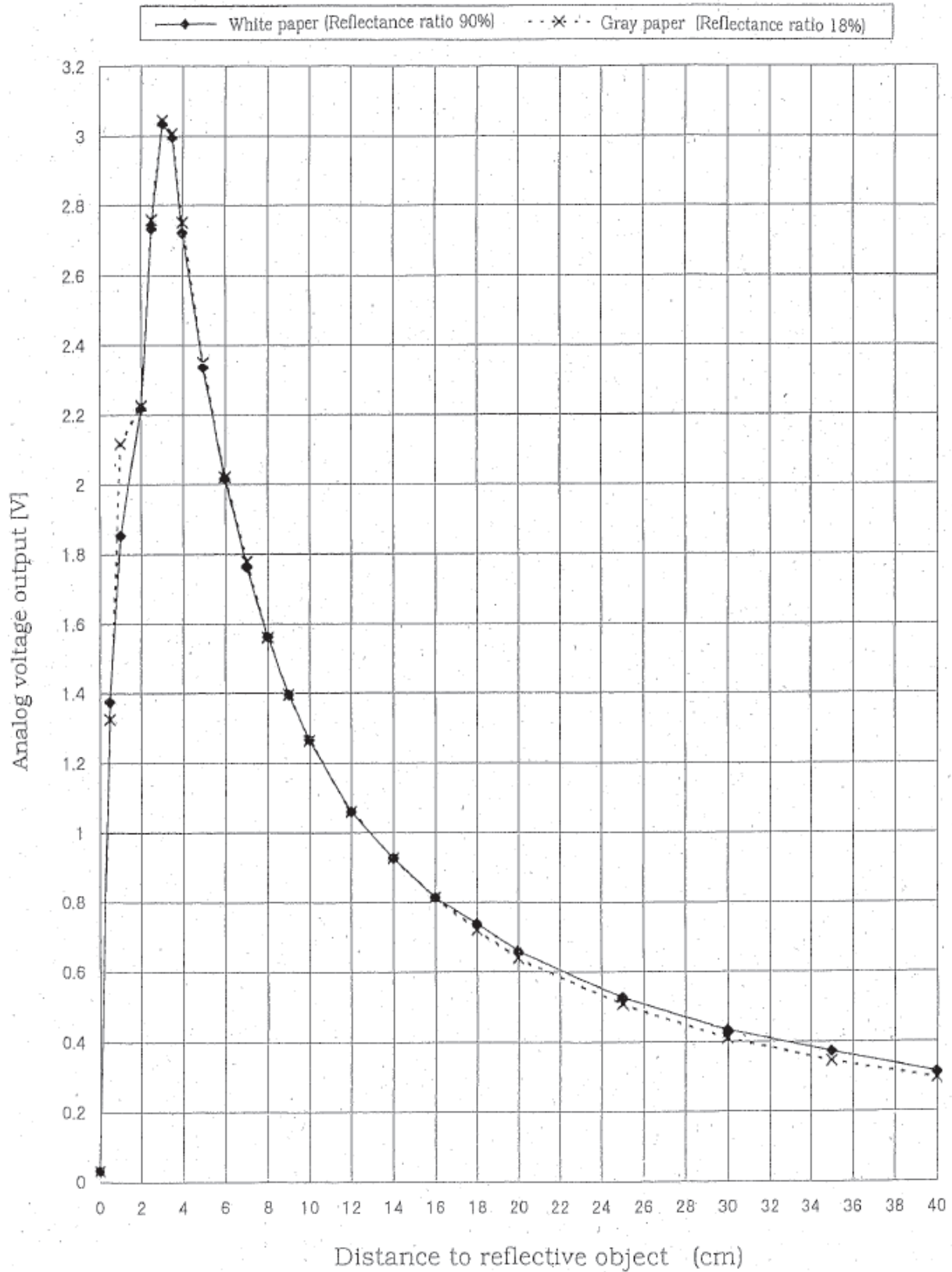


Abbildung 16b: Sharp Distanzsensor 31SK

3.15 FT-Akku

Der Fischertechnik-Akku dient allein zum testen der Motoren. Verbinden sie einen Motor mit dem Akku dreht sich dieser mit voller Geschwindigkeit.

Mit dem bereitgestellten Netzteil kann der Akku aufgeladen werden.



Abbildung 17: Fischertechnik Akku und Ladegerät

3.16 Carson-Akku

Der Carson-Akku kann mit der Gesamtplatine verbunden werden um diese mit Strom zu versorgen. Mit dem bereitgestellten Netzteil kann der Akku aufgeladen werden.



Abbildung 18: Carson-Akku und Ladegerät

3.17 Pololu Encoder Motor

Die Pololu Encoder Motoren sind wie in der nachfolgenden Tabelle anzuschließen.

Farbe	Anschluss
Rot	Motor 1, 2 ... 8
Schwarz	
Grün	GND

Blau	5V
Gelb	QEP A oder B
Weiß	

Die Signale des gelben und des weißen Ausgangssignals sind um 180° Phasenverschoben, wodurch neben dem zurückgelegten Winkel auch die Angabe der Drehrichtung erfolgen kann (Details hierzu gibt es über <https://www.pololu.com/product/2287>). Der Motor sowie der Encoder sind in Abbildung 19 dargestellt.

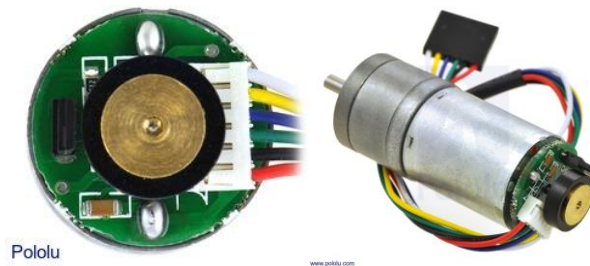


Abbildung 19: Polulu Encoder und Motor

3.18 Nexonar Tracker

Der Nexonar Tracker ist ein aktiver Sensor und wird zusammen mit dem Kamerasystem zur Positionsbestimmung verwendet. Der Tracker wird mit einem Schalter an der Seite angeschaltet, eine grüne Status-LED leuchtet auf. Mit selbigem Schalter wird der Tracker durch längeres drücken wieder ausgeschaltet. Ein Kabel zum Laden des Trackers liegt den Materialkästen bei. Über die seitlich angebrachten Ösen kann der Sensor befestigt werden. Tracker und Gesamtplatine einer Gruppe sind fest miteinander verknüpft. Die Position des Trackers im globalen Koordinatensystem wird für die „vordere“ Infrarot-LED, wie in Abbildung 20 mit einem roten Kreis hervorgehoben, angegeben.

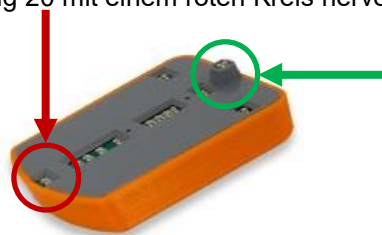


Abbildung 20: Nexonar Tracker

Die aktuelle Position des Trackers (falls dieser angeschaltet und in der Reichweite der Kameras ist) wird über einen separaten PC neben dem Spielfeld angezeigt. Weitere Informationen zu den ausgegebenen Werten und wie diese zum Mikrocontroller übertragen werden sind in Kapitel 5.5 erläutert. Die ID des Trackers ist neben der Gruppennummer auf dem Tracker angebracht und in der Tabelle am Ende dieses Abschnitts aufgeführt.

- ⚠ Zu beachten ist die erhöhte Infrarot-LED des Trackers (grüner Kreis in Abbildung 20). Diese Erhöhung wirkt unter Umständen einen Schatten, wodurch die Positionsbestimmung keine gültigen Werte erzeugt.
- ⚠ Überprüfen Sie die Bewegungen ihrer mechatronischen Systeme und passen Sie die Ausrichtung des Trackers entsprechend an.

Der Nexonar Tracker kann in den **Helligkeitsstufen 0, 20, 40, 60 ,80 oder 100** eingestellt werden. Je höher die Intensität der Infrarot-LEDs, desto besser werden diese vom Kamerasystem erfasst. Die Akkulaufzeit nimmt allerdings bei steigender Intensität ab.

- ⚠ Schalten Sie den Tracker also nur ein, wenn Sie diesen auch benutzen, damit die Ladekapazität des Akkus bestehen bleibt.

Zum Ändern der Helligkeitsstufe wird der Tracker über ein USB-Kabel an einen PC mit der Software **Nexonar Motion Visualizer** angeschlossen. Unter Windows 10 wird der Treiber zur Verbindung des Trackers mit dem PC automatisch installiert. Nach dem Öffnen des Programms wird ein beliebiges Projekt geöffnet oder ein neues Projekt angelegt. In den Einstellungen (Settings – Symbol: Zahnrad) sollte unter dem Reiter *Sensors* der angeschlossene Tracker angezeigt werden. Öffnen Sie die Einstellungen des Trackers über *Single Camera Tracker → Configure*. Die Helligkeitsstufe wird unter *Tracker Brightness → Set Tracker Brightness* eingestellt, mit *set → ok → ok* bestätigt.

- ⚠ Verändern Sie niemals die ID oder sonstige Einstellungen des Trackers!

Jeder Tracker hat eine ID. Diese ID ist fest mit der Gesamtplatine und der Gruppe verknüpft. Wenn Sie die Position ihres Markers bestimmen wollen, genügt es diesen in den Aufnahmebereich der Infrarot-Kameras zu halten und die entsprechenden Werte vom Monitor des Kamerasystem-PCs abzulesen (Falls der Bildschirmschoner oder der Energiesparmodus an ist → mit der Maus etwas hin und her wackeln). Durch die feste Verknüpfung ist es nicht möglich, dass alle Gruppen gleichzeitig die Position ihrer Tracker auslesen können.

- ⚠ Gruppen mit gleicher ID ihrer Tracker können nicht gleichzeitig mit dem Kamerasystem arbeiten (Beispiel: Gruppe 1b mit 6a oder 4a mit 5b).

Die Belegung der Tracker-IDs ist in der nachfolgenden Tabelle festgehalten.

Tabelle 1: Übersicht über die IDs der Gruppen, deren Tracker und die dazugehörige ID

Gruppe	ID	Tracker	Gruppe	ID	Tracker
1B	0	S03-0	1A	1	S04
2B	1	S04-1	2A	2	S05
3B	2	S05-2	3A	3	S07
4B	3	S07-3	4A	4	S09
5B	4	S09-4	5A	5	S10
6B	5	S10-5	6A	0	S03
7B	6	S12-6	7A	7	S13
8B	7	S13-7	8A	6	S01
9B	8	S15-8	9A	5	S16
10B	9	S16-9	10A	8	S08

4. Programmierung

In diesem Kapitel werden grundlegende Funktionen der Stateflow Toolbox von Matlab&Simulink erläutert und dargestellt.

4.1 Simulink Modell

Nachdem Matlab geöffnet ist, kann im linken Fenster ein Ordner ausgewählt oder neu erstellt werden, um zukünftige Simulink Modelle zu speichern. Im selben Fenster wird nach einem Rechtsklick *New File* → *Model* ein neues Simulink-Modell angelegt. Alternativ kann im Command Window „simulink“ eingegeben werden, um zuerst die Simulink Umgebung zu öffnen um dort ein neues Modell zu erstellen.

Die Einstellungen der Hardware Implementierung sind in den *Model Configuration Parameters* abgelegt. Die benötigten Blöcke sind der Modellbibliothek zu entnehmen: *Embedded Coder Support Package (...)* *Processors* → *F2837xD*.

- **ADC: Analog-Digital-Converter**, um analoge Messsignale in digitale Werte zu überführen.
- **Digital Input**: Zum Empfangen digitaler Signale (HIGH und LOW), z.B. zum Anzeigen eines geschlossenen oder geöffneten Schalters.
- **Digital Output**: Zur Ausgabe eines digitalen Signals (HIGH und LOW), z.B. die Drehrichtung eines Motors.
- **ePWM**: Zum Erstellen eines pulsweiten modellierten Signals, welches die Drehgeschwindigkeit eines Motors vorgibt.
- **eQEP**: Zum Auswerten der Positionswerte des Encoders der Polulu-Motoren.

Weitere Blöcke, z.B. logische oder arithmetische Operationen sind den Standardbibliotheken zu entnehmen.

4.2 Beispielprogramm I

Mit diesem Beispielprogramm kann das erfolgreiche Einstellen von Simulink, der Umgang mit Blöcken aus der Bibliothek und das Verbinden mit dem Mikrocontroller ohne angeschlossene Hardware getestet werden. Das Simulink-Modell ist in Abbildung 22 dargestellt.

Im ersten Schritt werden die Blöcke **Digital Output** und **Constant**, welcher in einer der Standardbibliotheken hinterlegt ist, in das Simulink Modell mit Drag-and-Drop eingefügt. Am rechten Rand des Constant Blocks wird mit Linksklick die Verbindung zum dahinterliegenden Digital Output Block hergestellt. Mit Doppelklick auf den jeweiligen Block, können seine Optionen und Konfigurationen abgerufen werden. Innerhalb des Constant Block können beispielsweise der Wert und die Sample Time eingestellt werden. Die Sample Time wird beispielhaft auf 0.5 geändert. Die Parameter der Embedded Coder Blöcke, hier Digital Output, beinhalten immer den Port des Mikrocontrollers an den das Signal der Software sowie die Hardware angeschlossen werden. Beispielhaft wird die Gruppe **GPIO32~GPIO39** gewählt. Neue Checkboxen erscheinen, ein Haken wird bei **GPIO32** und **Toggle GPIO32** gesetzt, das Fenster wird mit OK geschlossen.

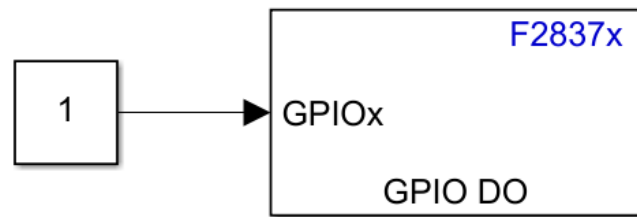


Abbildung 22: Beispielprogramm I

4.3 Programme auf den Mikrocontroller spielen

Um das erstellte Simulink-Programm auf dem Mikrocontroller auszuführen muss in Simulink unter dem Reiter „Hardware“ die Option „Monitor & Tune“ gewählt werden. Simulink beginnt dann einen ausführbaren C-Code zu erstellen, diesen zu compilieren und anschließend auf den Mikrocontroller zu flashen. Dieser Prozess kann mehrere Minuten dauern. Sobald der Prozess abgeschlossen ist, wird im unteren Bereich des Simulink-Fensters *Running the model on 'TI Delfino F28379D Launchpad'* angezeigt und Simulink wechselt in den Live-Modus. Wenn Sie in ihrem Programm „Scope“- oder „Display“-Blöcke aus den Standardbibliotheken verbaut haben so können Sie nun live die Werte verfolgen die der Mikrocontroller erzeugt. Außerdem besteht die Möglichkeit, während das Programm ausgeführt wird die Werte einzelner Blöcke zu ändern. Wenn Sie beispielsweise das Blinklicht-Programm aus Abschnitt 4.2 auf dem Mikrocontroller ausführen, können Sie den Wert der Konstante auf 0 setzen und die LED wird aufhören zu blinken. Wenn Sie weiter programmieren wollen, beenden Sie den Live-Modus durch drücken auf *Stop*.

- ⚠ Änderungen die Sie am Programm durchführen, während es auf dem Mikrocontroller ausgeführt wird gehen bei einem Neustart des Mikrocontrollers verloren. Wenn Sie Änderungen dauerhaft Anwenden wollen beenden Sie den Live-Modus und laden Sie das Programm erneut auf den Mikrocontroller.

4.4 Stateflow

Dieser Abschnitt gibt eine sehr kurze Einführung in das Thema Stateflow. Im Internet findet man ausreichend Literatur, FAQ's und weitere Informationen von Einsteiger- bis Expertenwissen. Die beigelegten Dateien *Stateflow GettingStarted.pdf* und *Stateflow UserGuide.pdf* sind gute Einsteiger- und Nachschlagedokumente. Mit der Stateflow Toolbox von Simulink wird es ermöglicht Simulink Modelle mit endlichen Zustandsautomaten (Finite State Machines) zu erweitern. Deren Zustände können dabei durch externe und interne Events oder Transitionen geschaltet werden. Die Toolbox ist im Simulink Library Browser unter Stateflow hinterlegt. Die beiden vorhandenen Blöcke *Chart* und *Truth Table* können mittels Drag-and-Drop in ein geöffnetes Modell gezogen und mit Doppelklick geöffnet

werden. Alternativ wird die Toolbox mit dem Befehl *stateflow* oder *sf* im Matlab Command Window aufgerufen.

4.5 Elemente eines Zustandsautomaten

Öffnet man einen erzeugten Automaten (Chart) gibt es mehrere Elemente in der linken Leiste des Fensters, die dazu genutzt werden können den Automatengraphen zu erstellen. Die wichtigsten sind nachfolgend kurz erläutert.

State: Innerhalb dieses Elements können Variablen manipuliert und arithmetische Operationen durchgeführt werden. Ein State (Zustand) wird immer aufgerufen, wenn eine Condition (Bedingung) einer Transition oder die Default Transition erfüllt ist.

Substate: Mit Drag-and-Drop können Zustände in einen vergrößerten Zustand gezogen werden. Den äußeren Zustand bezeichnet man als Superstate, alle inneren als Substates. Simulink gruppiert alle inneren Zustände wodurch ein verschieben des umfassenden Zustandes alle inneren Zustände automatisch mit ausrichtet. Die Substates werden nur aktiv wenn der äußere Zustand aktiviert wird. Dies ist ein sinnvolles Mittel um den Überblick zu bewahren.

Transition: Die Transition (Übergang) verbindet zwei Zustände, indem am Rand eines Zustandes mit Linksklick ein Pfeil zu einem weiteren Zustand gezogen wird. Zu jeder Transition wird mit Doppelklick auf den Pfeil eine Bedingung hinzugefügt, bei der der Zustand gewechselt wird. Notation: $[(condition)]$.

Junction: Die Junction (Verbindungsstelle) erlaubt mehrere Bedingungen beim Übergang eines Zustandes zu mehreren Anderen.

Default Transition: Diese Transition kennzeichnet den ersten Zustand des Automaten. Es kann ebenfalls eine Bedingung hinzugefügt werden. Wird keine Bedingung festgelegt startet der Automat beim Abspielen der Software direkt in diesen Zustand. Diese Transition muss auch bei einer Hierarchie durch Substates an einen Zustand angefügt werden.

Simulink/Matlab/Graphical Function: Sollen externe Funktionen von Matlab, Simulink Modelle oder grafische Funktionen in einem Zustand aufgerufen werden, sind diese zuvor durch den jeweiligen Block im Automaten zu definieren.

Weitere Elemente der Stateflow Toolbox sind dem *Stateflow UserGuide.pdf* zu entnehmen.

Ein On/Off Beispiel ist in Abbildung 23 dargestellt. Der Automat befindet sich am Anfang im Zustand On. Er wechselt in den zweiten Zustand Off, wenn die Variable $u := false$ gesetzt wird. Der Automat verbleibt im Zustand Off bis $u := true$ gesetzt wird und wechselt dann wieder in den Zustand On. Die Variable y wird genutzt um On ($y = 1$) und Off ($y = 0$) darzustellen. Am linken Rand sind die Symbole der vorhandenen Stateflow-Elemente zu sehen.

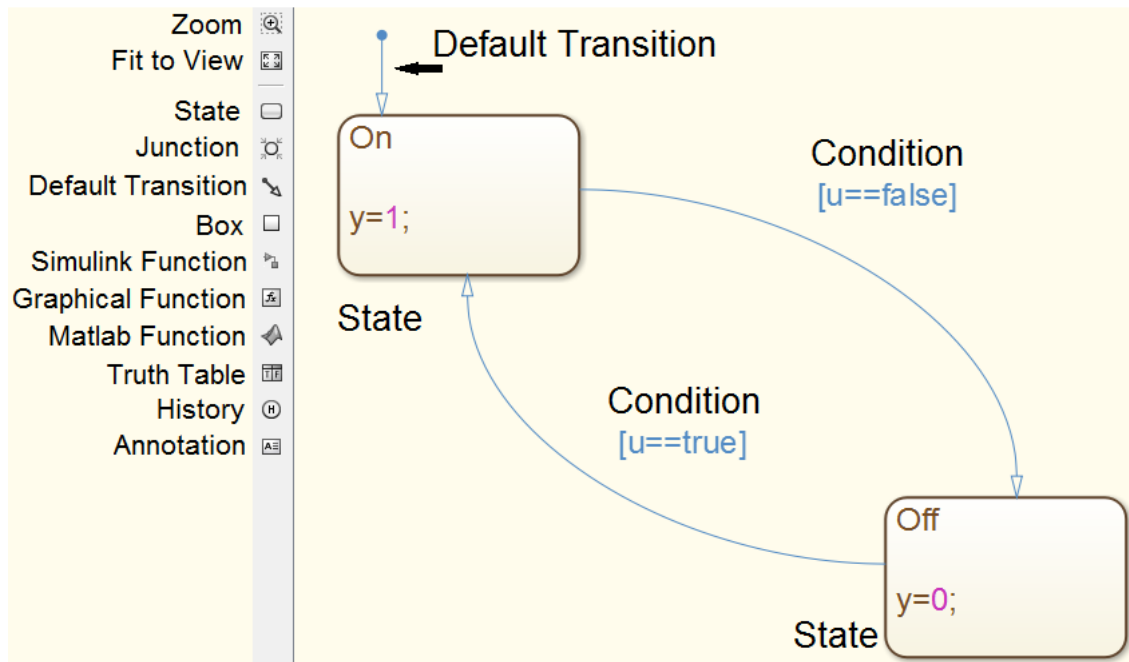


Abbildung 23: Stateflow On/Off Beispiel

Innerhalb eines Zustandes können mehrere Aktionen durchgeführt werden. Zusätzlich kann beim Wechsel in den Zustand die **entry** Operation, während des Zustands die **during** Operation und beim Austritt die **exit** Operation durchgeführt werden. Mit dem **bind** Befehl werden variablen und Ereignisse an den Zustand gebunden. Andere Zustände können diese Variablen lesen, aber nicht verändern. Eine gebundene Variable oder ein gebundenes Ereignis ist eindeutig. Außerdem gibt es den **on** Befehl, der ein Ereignis mit einer Aktion verbindet. Die Aktion wird ausgeführt, wenn der Zustand aktiv ist und das Ereignis eintritt. Notation:

```
entry: y=1;
during: y=y++;
exit: y=0;
bind: y;
on [Zustandsbezeichnung]:[Aktionsbezeichnung]; (Bsp: on sonnenuntergang:function_LichtAn();)
```

Ist dasselbe wie:

```
entry: y=1;
during: y=y++;
exit: y=0;
```

Sind dies alles Einträge eines Zustandes, wird beim Wechsel in diesen die Variable `y = 1` gesetzt. Für jegliches Ereignis wird diese Variable um Eins erhöht. Beim Wechsel in einen anderen Zustand wird `y = 0` gesetzt. Ebenfalls ist die Variable `y` an diesen Zustand gebunden. Nach kurzem Überlegen sollte man feststellen, dass jeder andere Zustand nur `y = 0` und nichts anderes erwarten oder nutzen kann. Befindet man sich noch in diesem Zustand und tritt das Ereignis *sonnenuntergang* ein, wird die Aktion, hier die Funktion `function_LichtAn()`, ausgeführt.

Eine sehr wichtige und nützliche Funktion ist die `after(n, E)` Anweisung die in einer Condition genutzt wird um nach einer definierten Zeit den Zustand zu wechseln. Sie kann also als Timer genutzt werden. Innerhalb der Anweisung gibt **n** einen numerischen Wert an und **E** die Einheit, z.B. sec, msec etc. Weitere Möglichkeiten sind der Stateflow Hilfe zu entnehmen.

4.6 Verbindung des Simulink Modells mit dem Automaten

Damit der Automat mit dem restlichen Simulink Modell kommunizieren kann, müssen ausgewählte Variablen als Input oder Output des Automaten definiert werden. Innerhalb des Automaten (Chart) mit Rechtsklick auf eine freie Stelle → *Explore* werden alle verwendeten Blöcke und Variablen des Modells angezeigt. Darunter ist auch der erstellte Automat, welcher ausgewählt wird. Möchte man nun für das Beispiel aus Abbildung 23 **u** als Eingang und **y** als Ausgang festlegen, werden zuerst zwei undefinierte Variablen über den Reiter *Add* → *Data* oder *Ctrl + D* hinzugefügt. Im mittleren Fenster werden beide Variablen aufgelistet. Wählt man eine aus, kann man im rechten Fenster den Namen, Variablentyp und die Richtung festlegen, z.B. **u**: Input, Boolean und **y**: Output, int32. Wird mit Ok bestätigt und verlässt man den Automaten, sieht man im Simulink Modell, dass der Chart Block mit einem Eingang **u** und einem Ausgang **y** erweitert wurde. Beliebige Simulink Syntax, die allerdings die Variablendeklaration des Automaten einhalten muss, kann nun über diese beiden Ports mit dem Automaten kommunizieren.

5. Software-Module

Die zur Verfügung gestellten Software-Module können über ILIAS heruntergeladen werden. Die Module befinden sich in innerhalb einer Datei, sowie zwei ergänzenden Dateien. Für die fehlerfreie Verwendung der Module müssen immer alle drei Dateien im selben Ordner abgelegt werden. Die einzelnen Module können aus der Datei herauskopiert und in einem anderen Modell eingebettet werden, solange die beiden Dateien `i2c_reset.c` und `i2c_reset.h` im selben Ordner sind. Verwenden Sie für neue Modelle bitte immer die zur Verfügung gestellte Vorlage, damit alle Hardware-Parameter korrekt eingestellt sind.

Matlab-Datei	Dateiendung
<code>i2c_reset.c</code>	<code>*.c</code>
<code>i2c_reset.h</code>	<code>*.h</code>
SoftwareModuleMatlab2021b	<code>*.slx</code>
VorlageMatlab2021b	<code>*.slx</code>

5.1 ADC-Kennlinie

Der Analog-Digital-Wandler des Mikrocontrollers bildet die analogen Gleichspannungen auf 4096 Werte ab, wobei ein linearer Spannungsteiler, zwischen der Digitalisierung und der Eingangsspannung, das Eingangssignal auf einen kleineren Wertebereich reduziert. Die Kennlinie zwischen digitalem und analogem Wert wird über den Block *ADC - Messwerte - Vorverarbeitung* dargestellt. Eine beispielsweise mit dem Multimeter gemessene Gleichspannung ADC_{in} entspricht der digital ausgegebenen Spannung

$U_{digital} = (ADC_{in} - \text{Offset}) * \frac{4.501}{4096}$. Der Block ist in Abbildung 27 dargestellt. Es ist zu beachten, dass die Kennlinien der Analog-Digital-Wandler leicht unterschiedlich sind, dies kann in der Praxis jedoch meist vernachlässigt werden.

- ⚠ Die Kennlinie der Eingänge **A_IN 1** und **A_IN 8** weichen teilweise von der angegebenen Kennlinie ab. Wenn Sie diese Eingänge verwenden wollen ist ggf. eine Neukalibrierung der Kennlinie erforderlich.

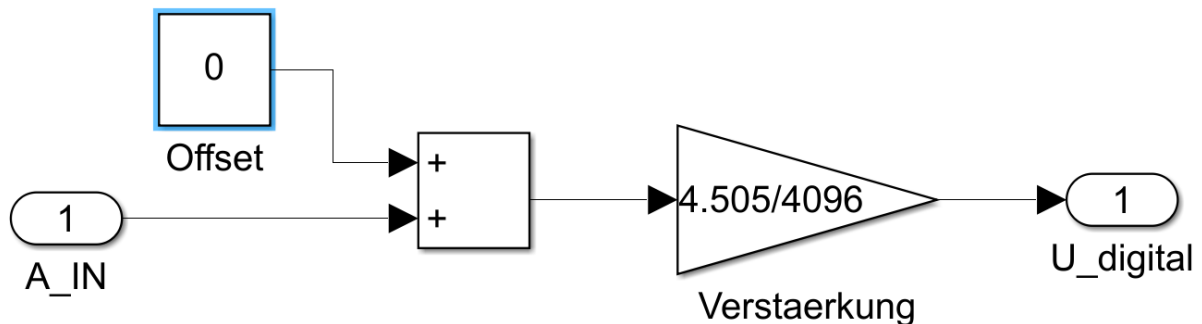


Abbildung 27: ADC - Verstärkung

5.2 Winkelmessung mit einen Pololu Encoder-Motor

Dieses Modul bestimmt den absoluten von einer Pololu-Encoder-Motorwelle verstrichenen Winkel sowie die aktuelle Drehrichtung. Der Pololu-Encoder-Motor ist dazu wie in Kapitel 3.17 beschrieben an zwei QEP-Eingänge anzuschließen.

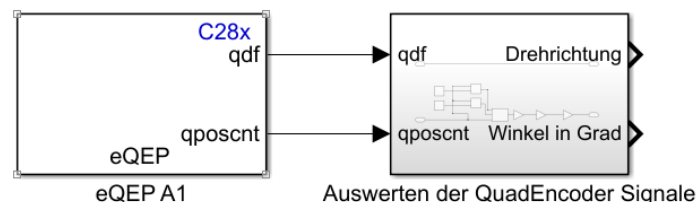


Abbildung 28: Messung des verstrichenen Winkels und der Drehrichtung eines Pololu-Encoder-Motors

5.3 Rate Transition

Dieser Block wird verwendet, um Programmteile mit unterschiedlichen Abtastzeiten miteinander zu verbinden. Dies bietet sich an, wenn Sie Messwerte von Sensoren mit einer hohen Abtastfrequenz abrufen wollen, aber daran anschließende Programmkomponenten nicht in der gleichen Geschwindigkeit abgearbeitet werden müssen. Eine geschickte Wahl der Abtastzeiten in Ihrem Programm, angepasst an die Anforderungen oder die fehlerfreie Funktionserfüllung der jeweiligen Programmteile, kann Ressourcen einsparen, wodurch die Abarbeitung des vollständigen Programms robuster geschieht.

- ⚠ Der Mikrocontroller ist kein echtzeitfähiges System! Es wird keine Warnung ausgegeben, wenn die Rechenressourcen überschritten wurden. D.h. im Falle eines zu aufwendigen Programmes entsteht eine Latenz und anschließende Programmteile werden verzögert ausgeführt.

Der *Rate Transition* Block befindet sich in der Standardbibliothek von Simulink.

5.4 P-Regelung

Innerhalb dieses Moduls findet ein klassischer Soll-Istwert-Vergleich statt. Der P-Anteil des Reglers k kann frei parametrisiert werden (Standardeinstellung $k = 0.1$). Weiterhin wird eine Totzone verwendet, um bei zu kleinen Regelungsabweichungen keine Stellgröße mehr zu erzeugen. Dies schont die Motoren und erzielt einen Stillstand innerhalb eines Bandes um den Sollwert (Standardeinstellung $[-100 \ 100]$). Innerhalb des Subsystems kann im Block *Begrenzung* der Wert der Ausgangs-PWM y angepasst werden. Werden keine Änderungen vorgenommen, variiert der Ausgangswert in Abhängigkeit vom Eingangswert (Betrag der Regelabweichung) zwischen $y = 0\%$ PWM und $y = 60\%$ PWM.

Die Ausgangssignale des Moduls können mit einem beliebigen Motor (Fischertechnik rot oder schwarz oder Polulu) verbunden werden. Verbinden Sie die Recht/Links-Drehung r/l_out sowie die PWM pwm_out mit einem entsprechenden Motor (Siehe Tabelle in Kapitel 3.6). Achten Sie auf die Reihenfolge der Anschlüsse, damit Sie keine Mitkopplung erzeugen. Das Modul ist in Abbildung 28 dargestellt.

Die Eingangssignale des Moduls können frei gewählt werden, der Istwert kann von einem beliebigen Sensor entnommen werden (z.B. der Positionsbestimmung) der Sollwert kann aus einem Zustandsautomaten resultieren und muss keine Konstante sein.

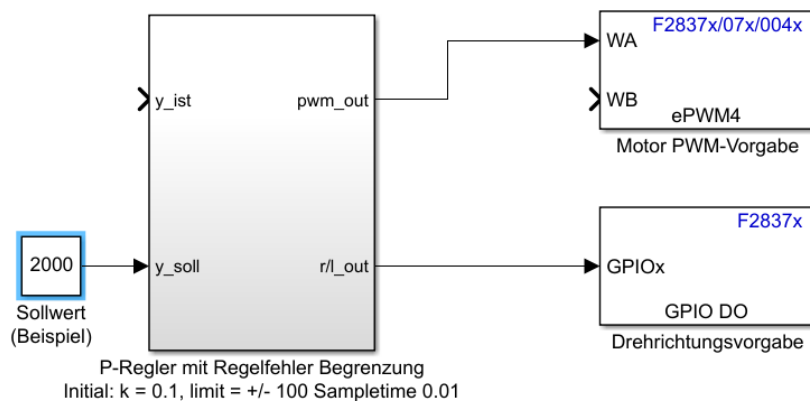


Abbildung 28: P-Regelung einer beliebigen Größe mit Stellsignal für einen Motor

5.5 Kommunikation mit anderen Systemen

In den Softwaremodulen ist noch der Block *Kommunikation* zu finden welcher in Abbildung 29 dargestellt ist. Dieser Block erfüllt zwei Funktionen:

1. Empfangen von aktuellen Positionsdaten von dem Nexonar-System
2. Senden und Empfangen von Nachrichten anderer Boards

Wie die beiden Funktionen verwendet werden und wie der Block entsprechend anzuschließen ist wird in folgenden erläutert.

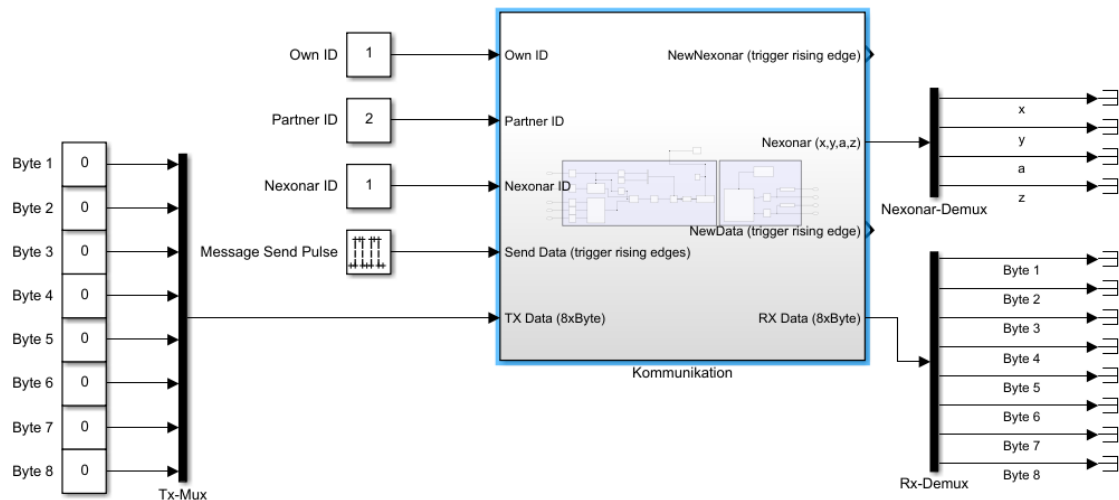


Abbildung 29: Modul *Kommunikation* sowie die Mux- und Demux-Blöcke für die Nexonar- und Sende/Empfangsdaten

Positionsbestimmung

Um die Positionsdaten Ihres Nexonar-Trackers zu empfangen müssen Sie zuerst in den Block *Nexonar-ID* die ID Ihres Nexonar-Trackers eingeben (siehe Abschnitt 3.18). Die Position des Nexonar-Trackers kann dann über die Ausgänge des Blocks *Nexonar-Demux* verarbeitet werden. Die Signale sind dabei die aktuellen Positionsdaten $[x_{ist} \ y_{ist} \ z_{ist}]$ im kartesischen Koordinatensystem sowie der Winkel α_{ist} (a) des Trackers in der xy -Ebene zwischen zwei auf dem Tracker befindlichen Infrarot-LEDs gegenüber der x -Achse.

⚠ Ändern Sie keine Einstellungen innerhalb dieses Blocks. Mit einer Abtastzeit von $\Delta t = 0.01 \text{ s}$ sollten die Positionswerte ausreichend schnell bereitgestellt werden.

Die bereitgestellten Werte sind nach folgendem Schema kodiert: Bei den angezeigten Werten (Kamerasystem-PC sowie digitaler Wert) muss eine Stelle von der letzten Ziffer beginnend ein Komma eingesetzt werden, um den Wert in mm zu erhalten.

Beispiel: $x = 1731$ (Kamerasystem-PC), dies entspricht $x_{ist} = 173,1 \text{ mm}$

Innerhalb des Moduls wird der Wert unverändert gegenüber der Anzeige auf dem Kamerasystem-PC übertragen. Vergleiche hierzu $y_{soll} = 2000$ aus Abbildung 28, dies entspräche $y = 200,0 \text{ mm}$. Der Soll-Istwert-Vergleich würde also mit dem unveränderten Wert durchgeführt werden. In der vollständigen Abarbeitungskette dieses Programms wird also keine Umrechnung in mm benötigt. Die Angabe in der Aufgabenstellung wird in beiden Maßen angegeben.

Der Winkel in der der xy -Ebene wird zwischen zwei LEDs auf dem Tracker gegenüber der x -Achse bestimmt. Die Winkelbeziehungen sind in Abbildung 30 zusammengefasst.

- ⚠ Testen Sie das Verhalten ihres Nexonar-Trackers in der Ebene und messen Sie die Winkel. Achten Sie dabei besonders auf die größere Ungenauigkeit im Bereich $+270^\circ \pm 20^\circ$ und den Sprung zwischen 360° zu 0° .
- ⚠ Positionieren Sie ihren Nexonar-Tracker möglichst Waagrecht in der xy -Ebene und versuchen Sie mit ihrem mechatronischen System eine *Verkippen* des Trackers zu vermeiden.

Die bereitgestellten Werte des Winkels α_{ist} sind nach folgendem Schema kodiert: Bei den angezeigten Werten (Kamerasystem-PC sowie digitaler Wert) muss eine Stelle von der letzten Ziffer beginnend ein Komma eingesetzt werden, um den Wert in $^\circ$ zu erhalten.

Beispiel: $\alpha = 223$ (Kamerasystem-PC), dies entspricht $\alpha_{ist} = 22,3^\circ$.

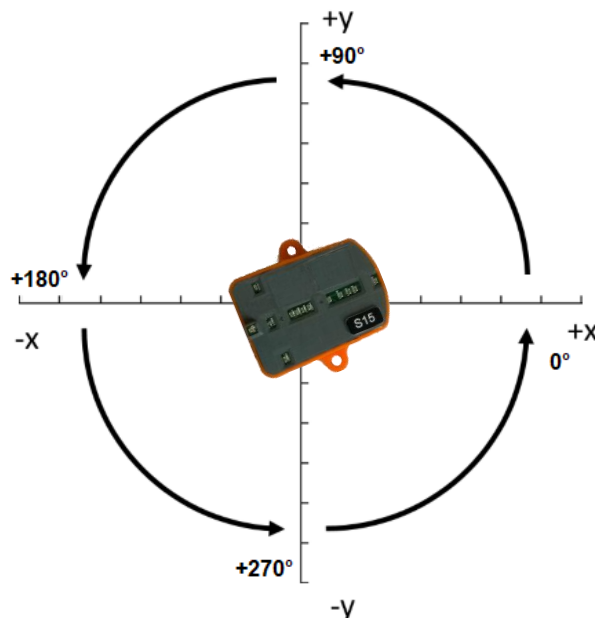


Abbildung 30: Winkelausgabe von α_{ist} (a). Beispielposition des Trackers im Bild: $\alpha_{ist} \approx 20^\circ$

Immer wenn sich die Positionswerte verändern, wird dies über eine steigende Flanke am Ausgang *NewNexonar* (*trigger rising edge*) angezeigt. Die Positionswerte stehen aber unabhängig davon immer am Ausgang des Blocks *Nexonar-Demux* zu Verfügung.

Kommunikation mit anderen Boards

Der Kommunikations-Block erlaubt außerdem eine Kommunikation mit anderen Boards (z.B. dem Board ihrer Partner-Gruppe). Wählen sie dazu eine eigene ID zwischen 0 und 255 und tragen diese in den Block *Own ID* ein. Tragen sie die ID des Boards mit welchem sie kommunizieren wollen (im Folgenden Partner-Board genannt) in den Block *Partner ID* ein.

- ⚠ Damit die Kommunikation korrekt funktionieren kann müssen die Werte *Own ID* und *Partner ID* unterschiedlich gewählt werden und müssen beide zwischen 0 und 255 liegen.

Über den Block *Message Send Pulse* können Sie festlegen, mit welcher Rate ihre Nachrichten verschickt werden sollen. Immer wenn an dem Eingang *Send Data (trigger rising edge)* eine steigende Flanke detektiert wird, wird eine Nachricht versendet. Die zu versendende Nachricht wird über den Eingang *TX Data (8xByte)* festgelegt. Der Eingang muss ein Vektor sein, welcher 8 Byte enthält. Dieser Vektor wird über den Block *Tx-Mux* erstellt, wobei jeder Eingang des Blocks *Tx-Mux* genau ein Byte darstellt.

- ⚠ Jeder Eingang des Blocks *Tx-Mux* muss ein ganzzahliger Wert zwischen 0 und 255 sein, andernfalls kommt es zu Fehlern.
- ⚠ Um sicherzustellen, dass Ihre Nachricht beim Partner-Board korrekt empfangen wird senden Sie bitte alle Nachrichten mehrfach hintereinander.

Immer wenn eine Nachricht von ihrem Partner empfangen wird, so wird diese als Vektor mit 8 Elementen über den Ausgang *RX Data (8xByte)* ausgegeben. Der Block *Rx-Demux* trennt den Vektor in 8 einzelne Ausgänge auf, wobei jeder Ausgang genau ein empfangenes Byte darstellt. Immer wenn neue Daten von dem Partner-Board empfangen wurden, wird dies über eine steigende Flanke an dem Ausgang *NewData (trigger rising edge)* angezeigt.